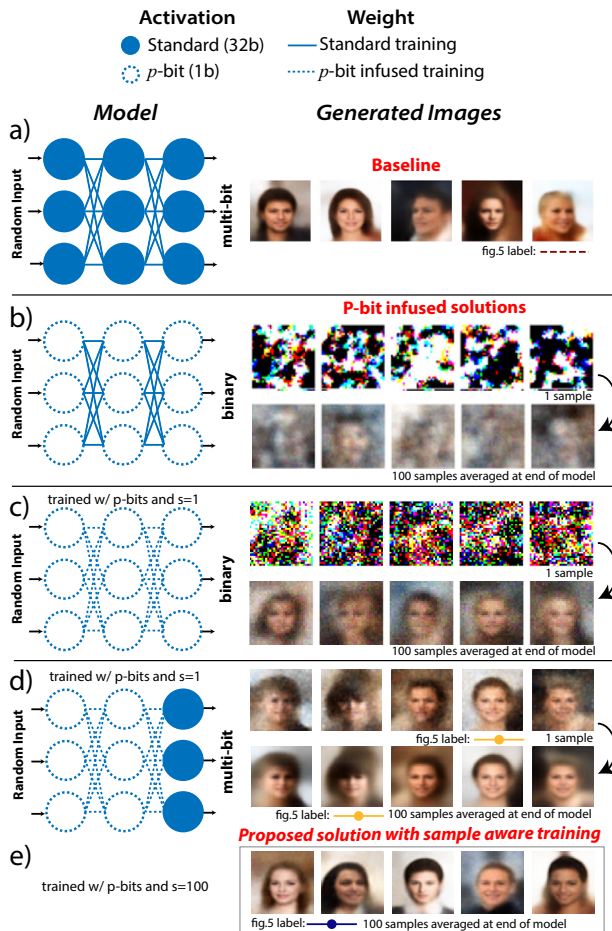




**Fig. 3 | Images generated via a traditional variational autoencoder compared with various probabilistic alternatives of the same architecture. a** standard DNN inference. **b** Model with pbits replacing all activations. **c** Model that is retrained with pbit activations. **d** Model that is retrained with pbit activations and an analog last layer. **e** Proposed sample aware training scheme employed on (d).

architectural considerations to take into account. For instance, the problems in ref. 1 use  $n = 6$ , which is relatively small so that the function  $W$  can be read off a look-up table (LUT) with  $2^n = 64$  entries. For p-bit building blocks where  $n$  is too large for the computation to be implemented with a LUT,  $\epsilon_S \sim n\epsilon_{add}(b_w)$ , where  $\epsilon_{add}(b_w)$  is the energy for adding two  $b_w$  bit numbers via an adder. For deterministic building blocks,  $\epsilon_S \sim n\epsilon_{mul}(b_a, b_w) + \epsilon_{add}(b_a)$ , where  $\epsilon_{mul}$  is the energy for multiplying a  $b_a$  bit neuron with a  $b_w$  bit weight. These products then need to be accumulated with the add.

**Neuron energy,  $\epsilon_N$ :** The neuron energy is defined as the energy needed to apply some non-linear function to an input. For p-bit based systems, the neuron may be implemented following the function  $b = \text{sign}(\tanh(W) - \text{rand}_{\{-1, +1\}})$  where  $\text{rand}_{\{-1, +1\}}$  is a random number distributed uniformly between  $-1$  and  $+1$ . The p-bit based neuron energy then has three sub-components: (1) compute the activation ( $\epsilon_{act}$ ), (2) generate  $\text{rand}_{\{-1, +1\}}$  ( $\epsilon_{RNG}$ ), and (3) compare the two to generate a binary output ( $\epsilon_{compare}$ ). Note that, unlike the memory read and compute energies, the sampling energy for p-bit based systems is independent of  $n$  and becomes relatively unimportant for large values of  $n$ , i.e., the neuron fan-in. In deterministic systems, there would be no RNG; for instance, in machine learning models, the neuron generally takes the form of some activation like ReLU or swiglu. For more complicated models, the neuron may take the form of softmax or normalization, which we will discuss later.

We define these components not to say that all probabilistic solutions should map to a single rigid formula, but rather to establish a framework with which to build custom energy functions for unique situations.

Figure 5 reveals that for low-K Boltzmann machine architectures, compute energy dominates the overall system. This motivates replacing the compute unit in these systems, which naturally requires sampling, with real p-bit hardware like s-MTJs or zener diodes to achieve meaningful end-to-end energy efficiency improvements. We will show in the coming sections that DNNs are not compute-dominated, but rather memory-dominated. We will argue that in such systems, minor increases in compute are almost negligible in the end-to-end pipeline in terms of energy, but can have a profound impact on the accuracy metrics of the task.

**65 nm ASIC for Quantum Monte-carlo.** Let us now discuss how the energy equation applies to a standard Boltzmann machine implemented with 65 nm CMOS technology<sup>1</sup>, which was used to solve a diverse set of problems having very different values of  $N$ , problem size, and  $T$ , sample requirement, but in each case the energy per elementary operation was about the same:  $\epsilon_p \sim 0.5pf$ . The energy split between the components of Eq. (1) is depicted in Fig. 5 for a specific configuration of hyperparameters pertaining to the QMC problem solved in ref. 1.

A fourth energy component acting as a design-specific catch-all term is added to Fig. 5 to include any control logic, I/O, clock distribution, routing, or other overheads. For the ASIC in ref. 1, this design-specific overhead amounted to 25% of the overall energy consumption, but we have not included this component in Eq. (1), since it is often not fundamental to the underlying algorithm or architecture.

Figure 5 shows that the Quantum-Monte Carlo problem is dominated by computational energy, where we can define compute as the sum of synapse and neuron energies. In such systems, it is clear that introducing additional compute would require careful consideration. In the next example, we will show that additional compute is much less costly for DNNs, and can be taken advantage of for notable accuracy gains.

**DNN's with sampling, p-DNN's.** The energy  $E$  for an  $N$ -neuron DNN is  $N$  times the energy per elementary operation ( $\epsilon_{EO}$ ) in Eq. (2):

$$E = N(nb_w \epsilon_{wM} + T(n+1)b_a \epsilon_{aM} + T\epsilon_S(n, b_a, b_w) + T\epsilon_N) \quad (3)$$

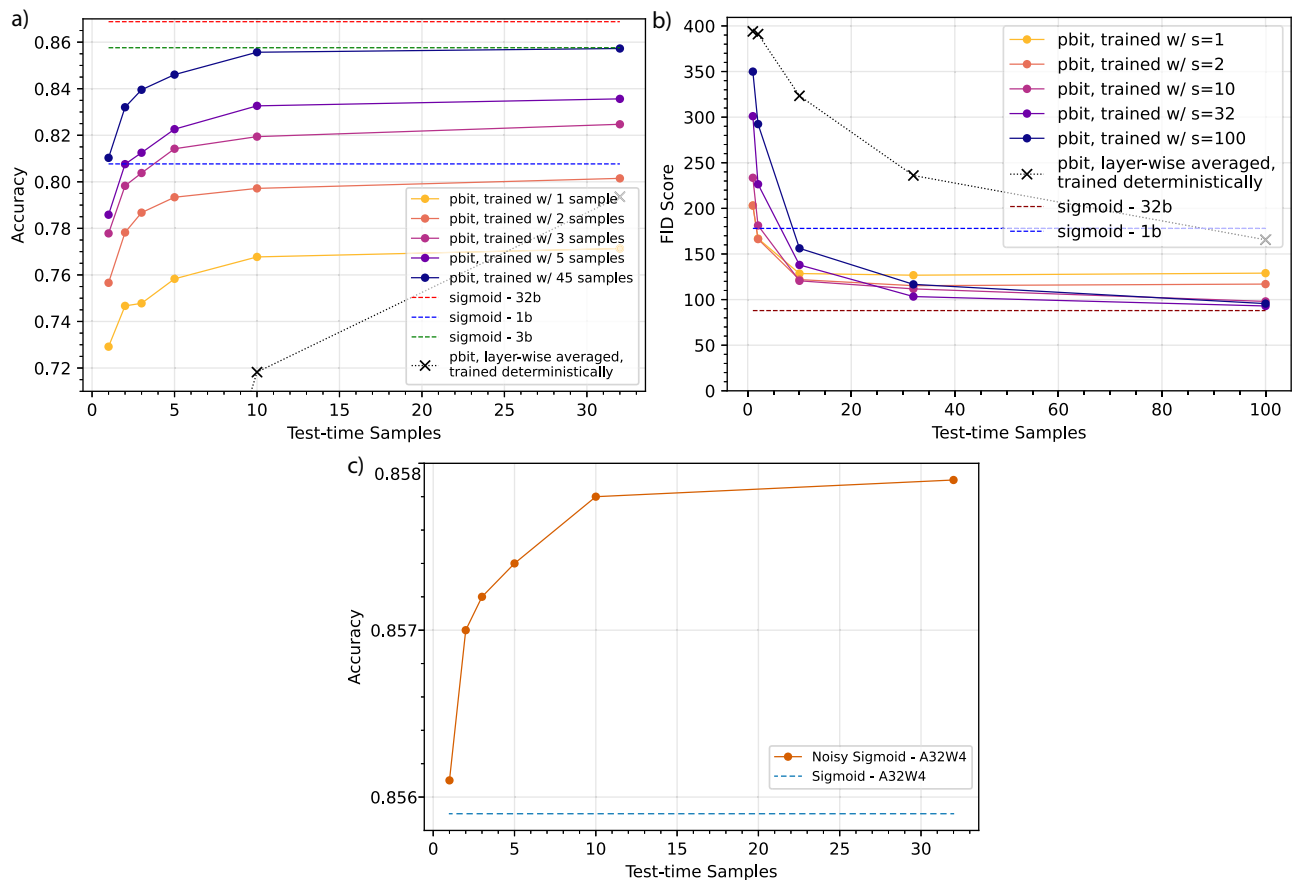
where a p-bit implementation could average over  $T$  samples to improve prediction quality. We bring  $T$  inside the elementary operation, as multiple samples can be generated without reloading the weights. This is why drawing multiple samples is significantly cheaper relative to drawing a single sample, where the weight loading energy cannot be amortized.

We can consider two cases for Eq. (3), a p-bit case and a standard case, where the p-bit case mimics the sample-aware training models (i.e., activations are replaced with p-bits). For the p-bit case,  $\epsilon_N \rightarrow \epsilon_{act} + \epsilon_{RNG} + \epsilon_{compare}$  and  $\epsilon_S \rightarrow n\epsilon_{add}$ , while for the deterministic case,  $\epsilon_N \rightarrow \epsilon_{act}$  and  $\epsilon_S \rightarrow n\epsilon_{mul} + n\epsilon_{add}$ . The energy gain can then be defined as  $gain = \frac{E_d}{E_p}$  where  $E_d$  is the energy draw for the deterministic DNN and  $E_p$  for the p-DNN. The  $n$  used is the average fan-in weighted by the number of parameters per layer in the VGG5 model in Fig. 9b. The standard DNN building block multiplies  $n$  weights (each  $b_w$  bits) with  $n$  activations (each  $b_a$  bits), while the p-bit implementation only sums up a selected subset of  $n$  weights (each  $b_w$  bits), corresponding to activations with  $b_a=1$ .

By plugging in the values for these quantities, recorded in Table 1 and visualized in Fig. 7, we can estimate the overall energy draw of a custom hardware built to run inference using these models, which is shown in Fig. 6. These quantities are simulated using the same platform and PDK used to design the ASIC in ref. 1. Note that this analysis compares p-DNNs with DNNs at the same weight bit width.

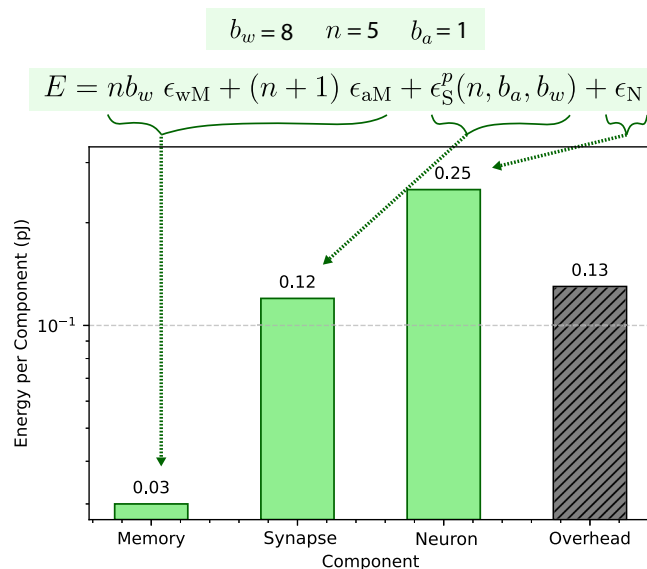
The following are some critical insights we can gather from this experiment.

- In contrast to the QMC chip results in Fig. 5, the memory energy dominates the overall ML pipeline, even for small sized models (~2M parameters), as seen in Fig. 6a. For applications requiring DRAM, which are most production scale LLMs today (100B+ parameter models), adding some compute for additional performance would be



**Fig. 4 | Quantitative performance evaluation of probabilistic and deterministic models on image classification and generation tasks. a** Comparison of accuracy for CIFAR10 classification. **b** Comparison of FID metrics for a variational

autoencoder trained on the CelebA dataset. **c** Accuracy on CIFAR10 of a deterministic baseline compared with a model with noisy activations.



**Fig. 5 | Energy cost of component operations of building block for QMC from 65 nm ASIC implementation reported in ref. 1.**

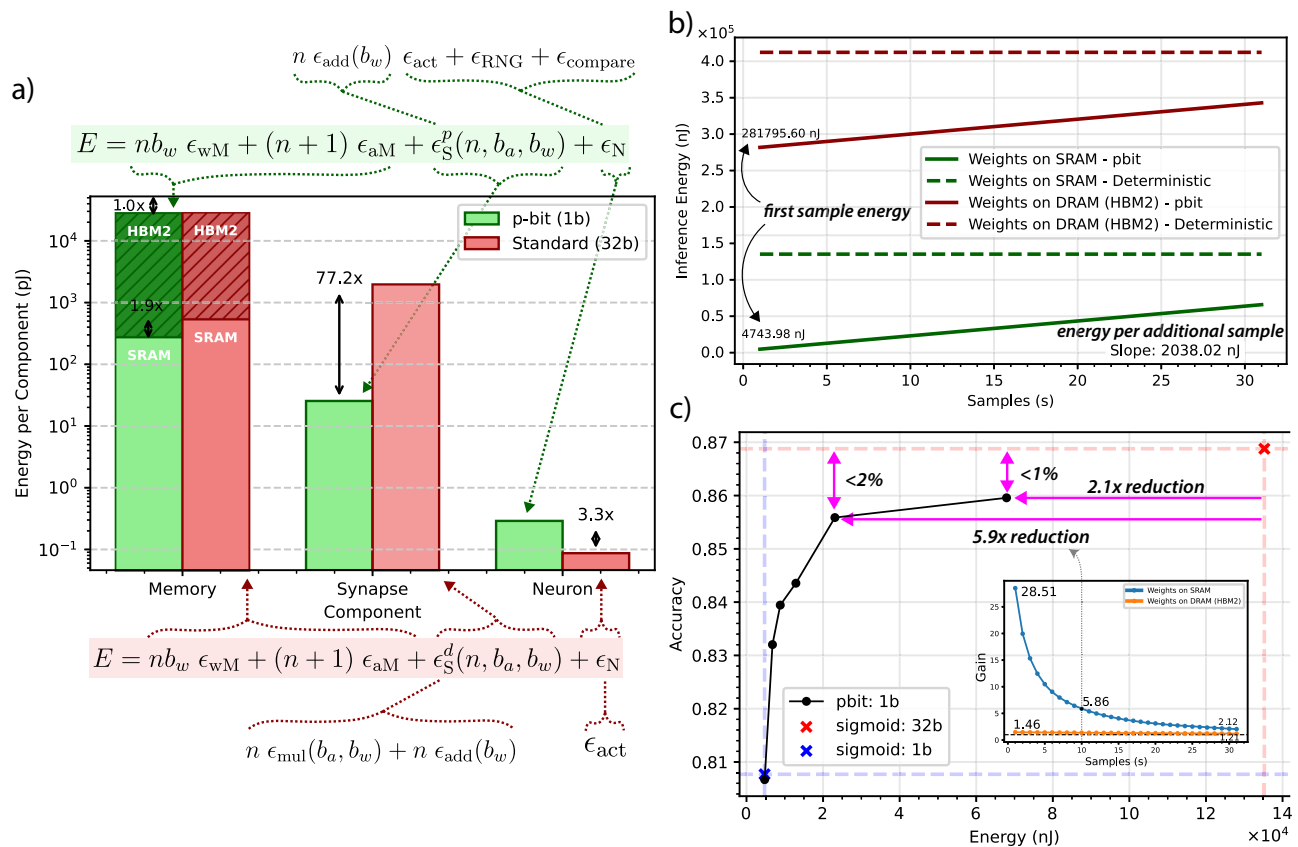
very desirable, as it would have little impact on the end-to-end energy cost of inference.

- As shown in Fig. 6a, p-DNNs using p-bit activations show dramatic improvement in synapse energy owing to the reduction of activations to 1 bit, the MAC to AC benefit.

**Table 1 | Variables for p-DNN analysis**

| Symbol               | Description                                | Quantities for Fig. 6                 |
|----------------------|--------------------------------------------|---------------------------------------|
| $N$                  | Number of neurons                          | 58,629                                |
| $n$                  | Fan-in to processing element               | 213                                   |
| $T$                  | Number of samples                          | 1                                     |
| $b_w$                | Bits per weight                            | 32                                    |
| $b_a$                | Bits per activation                        | 32, 1                                 |
| $\epsilon_{wM}$      | Energy to access 1b from weight memory     | $3.97 \frac{\text{pJ}}{\text{bit}}$   |
| $\epsilon_{aM}$      | Energy to access 1b from activation memory | $0.0384 \frac{\text{pJ}}{\text{bit}}$ |
| $\epsilon_{mul}$     | Energy to multiply $b_w$ and $b_a$         | 8.989 pJ                              |
| $\epsilon_{add}$     | Energy to add $b_w + b_a$ numbers          | 0.239 pJ                              |
| $\epsilon_{act}$     | Energy for activation                      | 0.087 pJ                              |
| $\epsilon_{rng}$     | Energy for RNG                             | 0.181 pJ                              |
| $\epsilon_{compare}$ | Energy for RNG-activation comparison       | 0.022 pJ                              |

- The first sample from a probabilistic DNN will be more costly than subsequent samples (see Fig. 6b). The energy cost of loading the weights from memory is amortized across multiple samples, but is felt entirely for 1-sample inference. Intermediate activation tensors will be stored back in activation memory in a  $T \times b_a$  memory chunk, and each layer will overwrite the same memory with new activation samples. Figure 6
- Figure 6c shows that tolerance for a 1% accuracy drop-off can come with a 2x+ reduction in energy used, going from a 32b DNN to p-DNN. We use accuracy per number of samples from the VGG5



**Fig. 6 | Energy cost analysis for probabilistic versus deterministic deep neural networks.** **a** Energy of component operations of building block for DNN p-bit and deterministic building blocks.  $n$  is the average number of inputs to a nonlinearity across all layers of the image generation DNN, scaled to match the dimensions of ResNet50. The SRAM components use  $\epsilon_{aM} = \epsilon_{wM} = 0.0384$  pJ/bit based on 1024-

word, 144-bits/word access to 65 nm TSMC process. The HBM2-DRAM components use  $\epsilon_{aM} = 3.97$  pJ/bit<sup>24</sup>, while retaining the activations on SRAM. **b** Shows energy per sample for the same setup as described for (a). **c** Combines energy per sample with accuracy per sample to show accuracy vs energy for the same setup as described for (a).

experiment in Fig. 4a and combine it with the energy per sample values in the inset of Fig. 6c to show an energy vs. accuracy curve comparing probabilistic and deterministic DNNs. From a purely computational perspective, though quantized models (like the 3b model matching performance of 10 p-bit samples) are competitive in energy iso-accuracy to stochastic models, it is important to remember that p-DNNs are *run-time adjustable*. More or fewer samples can be drawn at run-time to fit the application at hand, dynamically adjusting energy and accuracy to suit the user's needs.

### Expanding on the framework

The real power of this framework is how easily it can be tweaked and expanded on as needed. For example, this framework can be tweaked to find the energy per inference for an entire model, even for models that may not traditionally be thought of as breaking down neatly into *synapse* + *neuron* elementary operations. For instance, Large Language Models (LLMs) like Llama have various nonlinearities in addition to more traditional neurons like the SiLU, including but not limited to RMSNorm, softmax, and ROPE. The last term ( $\epsilon_N$ ) of Eq. (1) can be split into multiple neuron types as needed to cover these various nonlinearities. The energy for the entire model can be defined by scaling weight reads by the number of weights across the model, scaling synapse energy by the number of macs across the model, and scaling each neuron type's energy by the number of those neurons in the overall model.

From Eq. (1) then, one would arrive at the following function for the energy per frame of inference (or equivalently for an LLM, Joules/Token):

$$E_{\text{frame}} = n_{\text{weights}} b_w \epsilon_{wM} + T(n_{\text{mac}} (b_a \epsilon_{aM} + E_{\text{synapse}}) + n_{\text{neurons}_1} (E_{\text{neuron}_1} + b_a \epsilon_{aM}) + \dots + n_{\text{neurons}_N} (E_{\text{neuron}_N} + b_a \epsilon_{aM})) \quad (4)$$

where there may be  $N$  different types of nonlinearities in the model. One may want to test a model where different activations are stored in different memories, i.e.  $\epsilon_{aM}$  could be unique to each neuron type. This could be applied to the KV cache, for instance, which might be larger or further from compute than other activation buffers. The granularity with which an estimation can be made can be adjusted as desired.

### Comparing FPGA implementations of DNN and p-DNN

We implement both a P-DNN with p-bit activations and a traditional DNN with sigmoid activations in an FPGA to validate the energy framework. FPGA-based hardware implementations of probabilistic models outside of stochastic computing approaches have dealt exclusively with energy-based models<sup>6–8</sup>. On the other hand, this is not an energy-based application, though the general architecture is intentionally kept very similar to existing p-bit hardware solutions, as seen in Fig. 7. In keeping the design similar to existing p-bit hardware solutions, a few key decisions were taken: